

Transient (Selective Serialization)

Selective serialization means choosing which fields of an object should be saved during the serialization process and which should be excluded. The `transient` keyword in Java is used to achieve this.

👉 **transient** Keyword

When you mark a field as `transient`, you're telling Java to skip this field during serialization. This means:

- The field will not be written to the output file during serialization
- When deserialized, the field will be initialized with its default value (0 for integers, null for objects, etc.)
- Useful for data that doesn't need to be saved or shouldn't be saved (like passwords, temporary calculations, etc.)

Example:

The Student class

```
class Student implements Serializable {
    private int id;
    private String name;
    transient private int age; // The age field is marked as transient

    public Student(int id, String name, int age) {
        this.age = age;
        this.id = id;
        this.name = name;
    }

    public void disp() {
        System.out.println("id "+id);
        System.out.println("name "+name);
        System.out.println("age "+age);
    }
}
```

- Notice that only the age field has the `transient` keyword. This means id and name will be serialized, but age will not.

Serialization Process

```
public class LaunchIO {
    public static void main(String[] args) {
        // Create a Student object
        Student stu1 = new Student(1, "Rohan", 16);
        stu1.disp(); // Display object details before serialization

        try {
            // Create file output stream
            FileOutputStream fos = new FileOutputStream("D:\\telusko\\InputOutput\\serial.txt");

            // Create object output stream
            ObjectOutputStream oos = new ObjectOutputStream(fos);

            // Serialize the object
            oos.writeObject(stu1); // All fields except age are serialized

            System.out.println("Please check the file to see serialized object");

            // Close streams
            oos.close();
            fos.close();
        } catch (FileNotFoundException ex) {
            ex.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
id 1
name Rohan
age 16
Please check the file to see serialized object
```

Deserialization Process

```
public class LaunchIO {
    public static void main(String[] args) {
        try {
            // Create file input stream
            FileInputStream fis = new FileInputStream("D:\\telusko\\InputOutput\\serial.txt");

            // Create object input stream
            ObjectInputStream ois = new ObjectInputStream(fis);

            // Deserialize the object
            Student student = (Student) ois.readObject();

            // Display the deserialized object
            student.disp();

            // Close streams
            ois.close();
            fis.close();
        } catch (FileNotFoundException ex) {
            ex.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        } catch (ClassNotFoundException ex) {
            ex.printStackTrace();
        }
    }
}
```

Output:

```
id 1
name Rohan
age 0
```

👉 Explanation

- We serialized a **Student** object with values: id=1, name="Rohan", age=16
- During serialization, the **age** field was ignored because it was marked as **transient**
- When we deserialized the object, the **id** and **name** fields were restored with their original values
- The **age** field was set to its default value (0 for integers) since it wasn't part of the serialized data

Remember

- Fields marked as `transient` are excluded from serialization
- Transient fields are reset to their default values during deserialization
- Static fields are also not serialized (but for different reasons than transient fields)
- You can manually handle transient fields by implementing `writeObject()` and `readObject()` methods
- Use `transient` for any data that doesn't make sense to save or that you want to exclude for security or performance reasons